Análise Empírica de Algoritmos de Ordenação: Uma Avaliação Experimental de Quick Sort, Merge Sort, Heap Sort e Radix Sort em Entradas de Grande Volume

Gustavo R. Mazur[1]

Departamento de Informática – Universidade Tecnológica Federal do Paraná (UTFPR)
gustamomazur@alunos.utfpr.edu.br

Abstract

Sorting algorithms with sub-quadratic time complexity are fundamental to modern computing, yet algorithms within the same asymptotic class exhibit substantially different practical behavior due to memory access patterns, cache utilization, and pivot or partitioning strategies. This work presents an empirical study comparing four sorting algorithms — Quick Sort, Merge Sort, Heap Sort, and Radix Sort — through a reproducible benchmarking infrastructure implemented in Go 1.23. Experiments were conducted on five input sizes (300,000 to 2,500,000 elements) under three dataset configurations (sorted, reversed, and random), with three independent runs per combination. Metrics collected include execution time, elementary operations (comparisons and element movements), and heap memory allocations. The results confirm theoretical complexity predictions and reveal relevant practical distinctions: Radix Sort, a non-comparison-based algorithm, consistently outperforms all others by circumventing the $\Omega(n \log n)$ lower bound; Quick Sort with median-of-three pivot selection achieves its fastest execution on sorted inputs; Heap Sort incurs the highest execution time despite sharing the same asymptotic class as Quick Sort and Merge Sort due to poor cache locality; and Merge Sort achieves maximum timing stability at the cost of significant auxiliary memory allocation.
Keywords: Sorting Algorithms, Empirical Analysis, Computational Complexity, Quick Sort, Radix Sort, Go Language.

Resumo

Algoritmos de ordenação com complexidade de tempo sub-quadrática são fundamentais para a computação moderna, porém algoritmos dentro da mesma classe assintótica exibem comportamento prático substancialmente distinto em razão de padrões de acesso à memória, localidade de *cache* e estratégias de particionamento ou seleção de pivô. O presente trabalho apresenta um estudo empírico comparando quatro algoritmos de ordenação — Quick Sort, Merge Sort, Heap Sort e Radix Sort — por meio de uma infraestrutura de *benchmarking* reprodutível implementada em Go 1.23. Os experimentos foram conduzidos sobre cinco tamanhos de entrada (300.000 a 2.500.000 elementos) sob três configurações de *dataset* (ordenado, invertido e aleatório), com três execuções independentes por combinação. As métricas coletadas incluem tempo de execução, operações elementares (comparações e movimentações de elementos) e alocações de memória em *heap*. Os resultados confirmam

as predições teóricas de complexidade e revelam distinções práticas relevantes: o Radix Sort, único algoritmo não-comparativo avaliado, supera consistentemente os demais ao contornar o limite inferior $\Omega(n \log n)$; o Quick Sort com seleção de pivô pela mediana de três atinge desempenho máximo em entradas ordenadas; o Heap Sort apresenta o maior tempo de execução apesar de compartilhar a mesma classe assintótica dos demais, em razão de baixa localidade de *cache*; e o Merge Sort alcança máxima estabilidade temporal ao custo de alocações de memória auxiliar significativas.
Palavras-chave: Algoritmos de Ordenação, Análise Empírica, Complexidade Computacional, Quick Sort, Radix Sort, Linguagem Go.

1 Introdução

A escolha do algoritmo de ordenação adequado é uma decisão recorrente em engenharia de software, impactando desde o desempenho de sistemas de banco de dados até a eficiência de pipelines de processamento de dados em larga escala [2]. Embora a análise assintótica forneça um primeiro critério de seleção, algoritmos pertencentes à mesma classe de complexidade $O(n \log n)$ podem divergir substancialmente em desempenho real em função de fatores que a notação $O(\cdot)$ abstrai deliberadamente: localidade de *cache*, custo de alocação de memória, qualidade da seleção de pivô e sensibilidade à distribuição de entrada [5].

Além dos algoritmos comparativos de classe $O(n \log n)$, existe uma família distinta de algoritmos baseados em contagem ou distribuição — como o Radix Sort — que contorna o limite inferior $\Omega(n \log n)$ provado para ordenação por comparação [2]. Esses algoritmos exploram propriedades estruturais dos dados (como a representação numérica dos elementos) para atingir complexidade $O(nk)$, onde k é o número de dígitos da chave. Essa diferença fundamental requer avaliação empírica para quantificar o ganho prático em cenários reais.

Diante desse contexto, o presente trabalho adota uma abordagem experimental e reprodutível para comparar quatro algoritmos de ordenação amplamente empregados. Especificamente, os objetivos são:

1. Implementar e instrumentar os algoritmos Quick Sort, Merge Sort, Heap Sort e Radix Sort com contagem exata de comparações e movimentações de elementos;
2. Executar os algoritmos sobre três tipos de *dataset* (ordenado, invertido e aleatório) em cinco tamanhos de entrada (300.000 a 2.500.000 elementos);
3. Comparar os resultados empíricos com as predições teóricas de complexidade;
4. Identificar e discutir divergências práticas não capturadas pela análise assintótica, em particular a influência da distribuição de entrada e dos padrões de acesso à memória.

Os algoritmos foram selecionados por representarem estratégias distintas de ordenação: o Quick Sort baseia-se em particionamento por pivô; o Merge Sort aplica a

estratégia de divisão e conquista com fusão; o Heap Sort utiliza uma estrutura de *heap* binário; e o Radix Sort emprega distribuição por dígito, dispensando comparações entre elementos.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico dos algoritmos avaliados; a Seção 3 discute os trabalhos relacionados; a Seção 4 descreve a metodologia experimental adotada; a Seção 5 apresenta os resultados obtidos; a Seção 6 analisa e discute esses resultados; e a Seção 7 conclui o artigo.

2 Referencial Teórico

2.1 Quick Sort

O Quick Sort, proposto por Hoare em 1962 [4], é um algoritmo de ordenação baseado em divisão e conquista que seleciona um elemento *pivô* e reorganiza o vetor de modo que todos os elementos menores que o pivô precedam-no e todos os maiores sucedam-no. O processo é aplicado recursivamente às duas subpartições resultantes.

A complexidade de tempo do Quick Sort é $O(n \log n)$ no caso médio e $O(n^2)$ no pior caso, que ocorre quando partições degeneradas são geradas sistematicamente — por exemplo, ao selecionar sempre o primeiro ou último elemento como pivô em vetores já ordenados [2]. A seleção do pivô pela *mediana de três* — o elemento mediano entre o primeiro, o central e o último do subvetor — mitiga eficazmente esse risco, garantindo partições equilibradas mesmo em entradas ordenadas ou invertidas [8]. A complexidade de espaço é $O(\log n)$ em virtude da pilha de recursão, não havendo alocação de memória auxiliar proporcional a n.

2.2 Merge Sort

O Merge Sort é um algoritmo de divisão e conquista que divide recursivamente o vetor em dois sub-vetores de tamanhos aproximadamente iguais, os ordena de forma independente e, em seguida, os funde (*merge*) em um único vetor ordenado [2]. A complexidade de tempo é $O(n \log n)$ para todos os casos (melhor, médio e pior), uma vez que o número de níveis de recursão é $\lceil \log_2 n \rceil$ e cada nível realiza $O(n)$ trabalho na fusão [5].

Uma característica fundamental do Merge Sort é sua estabilidade: elementos com chaves iguais mantêm a ordem relativa original do vetor de entrada. A implementação de fusão requer um vetor auxiliar de tamanho $O(n)$, o que implica complexidade de espaço $O(n)$ — distinção relevante em sistemas com restrições de memória [3].

2.3 Heap Sort

O Heap Sort, introduzido por Williams em 1964 [9], é um algoritmo de ordenação baseado em uma estrutura de dados de *heap* binário máximo (*max-heap*). O algoritmo opera em duas

fases: primeiro, constrói o *heap* a partir do vetor de entrada (*heapify*), em $O(n)$; depois, extrai repetidamente o elemento máximo e o posiciona ao final do vetor, realizando $O(\log n)$ operações de restauração do *heap* por extração.

A complexidade de tempo é $O(n \log n)$ para todos os casos, e a complexidade de espaço é $O(1)$, pois o algoritmo opera *in-place* sem estruturas auxiliares adicionais [2]. Apesar dessas garantias teóricas, o Heap Sort é reconhecidamente penalizado por baixa localidade de *cache* em hardware moderno: o acesso ao *heap* binário segue um padrão de saltos não sequenciais na memória (de i para $2i + 1$ ou $2i + 2$), que conflita com a hierarquia de *cache* e a lógica de *prefetch* dos processadores modernos [1].

2.4 Radix Sort

O Radix Sort é um algoritmo de ordenação baseado em distribuição que não realiza comparações diretas entre elementos [2]. A variante LSD (*Least Significant Digit*) processa o vetor em k passagens, onde k é o número de dígitos da maior chave, ordenando por dígito do menos para o mais significativo. Em cada passagem, os elementos são distribuídos em baldes (*buckets*) de acordo com o valor do dígito corrente e, em seguida, coletados em ordem.

A complexidade de tempo é $O(nk)$, onde n é o número de elementos e k é o número de dígitos na representação da chave. Quando k é constante ou $O(\log n)$, o algoritmo supera os algoritmos comparativos, que estão limitados inferiormente por $\Omega(n \log n)$ [5]. A complexidade de espaço é $O(n + b)$, onde b é a base numérica utilizada (tipicamente $b = 10$ ou $b = 256$), em virtude da alocação dos vetores de contagem e do vetor de saída. O Radix Sort não é diretamente aplicável a dados não-inteiros sem transformação prévia.

3 Trabalhos Relacionados

A literatura sobre análise experimental de algoritmos de ordenação pode ser organizada em dois grupos principais: estudos empíricos comparativos de desempenho e análises específicas da interação entre algoritmos e características de hardware.

Estudos Empíricos Comparativos de Desempenho

Pizarro-Vasquez et al. [7] conduziram uma avaliação empírica de onze algoritmos de ordenação — incluindo Quick Sort, Merge Sort, Heap Sort e Shell Sort — implementados em C e Python. Os experimentos utilizaram quatro configurações de entrada (aleatória, quase ordenada, invertida e com poucos valores únicos) e avaliaram tempos de execução. Os autores confirmaram que algoritmos de classe $O(n \log n)$ superam consistentemente os de classe $O(n^2)$ à medida que o tamanho da entrada cresce. Contudo, o estudo não instrumenta contagens exatas de operações elementares, não quantifica a variabilidade estatística entre execuções independentes e não avalia algoritmos não-comparativos como o Radix Sort.

Musser [6] propôs o Introsort, algoritmo híbrido que combina Quick Sort, Heap Sort e Insertion Sort para garantir $O(n \log n)$ no pior caso com constante prática reduzida. A análise demonstra que o Heap Sort, embora assintoticamente equivalente ao Quick Sort, impõe sobrecarga significativa em cenários de pivô degenerado. Esse trabalho evidencia que a diferença prática entre algoritmos da mesma classe assintótica é suficientemente grande para motivar soluções híbridas em implementações de produção — motivação direta para o estudo empírico conduzido no presente artigo.

Análise de Predição de Desvios e Localidade de Cache

Biggar et al. [1] investigaram o comportamento de algoritmos de ordenação clássicos sob a perspectiva de predição de desvios em processadores modernos. Os autores demonstraram que algoritmos com mesma complexidade assintótica podem divergir substancialmente em desempenho prático em função do *miss rate* do preditor de desvios e da localidade de *cache*. Em particular, o estudo evidencia que o Heap Sort apresenta o pior desempenho de *cache* dentre os algoritmos $O(n \log n)$, enquanto o Merge Sort, apesar de alocar memória auxiliar, apresenta padrão de acesso sequencial favorável ao *prefetch* do processador.

Diferentemente dos trabalhos acima, o presente artigo adota instrumentação *in-code* que contabiliza exatamente as comparações e movimentações realizadas por cada algoritmo, incluindo o Radix Sort — um algoritmo não-comparativo ausente na maioria dos estudos experimentais identificados. A infraestrutura de *benchmarking* é implementada em Go — linguagem que compila para código de máquina nativo, sem *overhead* de JIT — e avalia entradas de até 2.500.000 elementos com três execuções independentes por combinação, permitindo quantificar a variabilidade das medições.

4 Metodologia

4.1 Ambiente Experimental

Todos os experimentos foram executados em hardware dedicado, sem virtualização, conforme descrito na Tabela 1. O sistema operacional não foi reiniciado entre as execuções, mas o *benchmarking* foi isolado por invocações explícitas do Garbage Collector antes de cada medição.

4.2 Implementação e Instrumentação

A linguagem Go foi escolhida por ser compilada estaticamente para código de máquina nativo (x86-64), eliminando o *overhead* de interpretação e o *warm-up* de compiladores JIT, presentes em linguagens como Java e C#. O tempo medido reflete, portanto, o custo computacional real das operações elementares.

Table 1. Especificações do ambiente de experimentação.

Componente	Especificação
Modelo	Dell G15 5515
Processador	AMD Ryzen 5 5600H, 3,3 GHz base, 6 núcleos / 12 threads
Arquitetura	x86-64
Memória RAM	16 GB DDR4
Sistema Operacional	Windows 11 Home (Build 26200)
Linguagem	Go 1.23 (compilação nativa, sem JIT)

Cada algoritmo foi implementado satisfazendo a interface `SortAlgorithm`, que exige os métodos `Name() string` e `Sort(arr []int) SortResult`. A estrutura `SortResult` encapsula o vetor ordenado, o total de comparações e o total de movimentações de elementos, contados incrementalmente durante a execução — instrumentação *in-code* sem interceptação externa.

O Quick Sort foi implementado com seleção de pivô pela mediana de três elementos (primeiro, central e último do subvetor), garantindo partições equilibradas em entradas ordenadas e invertidas. O Merge Sort utiliza fusão com vetor auxiliar alocado por operação de fusão. O Radix Sort utiliza a variante LSD (*Least Significant Digit*) em base decimal, com número de passagens determinado dinamicamente pelo valor máximo do vetor de entrada. O Heap Sort implementa o esquema clássico de *heapify* descendente com extração do máximo.

O motor de *benchmarking* adota as seguintes garantias de isolamento:

- **Cópia limpa de dados:** cada execução recebe uma cópia independente do *dataset* original, evitando que a ordenação in-place de uma execução afete as subsequentes.
- **Limpeza do Garbage Collector:** duas chamadas explícitas a `runtime.GC()` antes de cada medição eliminam o ruído de coletas de ciclos anteriores.
- **Medição de tempo de alta resolução:** `time.Since(start)` utiliza o relógio monotônico do sistema operacional com resolução de nanossegundos.
- **Medição de memória:** `runtime.MemStats` é capturado antes e após cada ordenação, contabilizando as alocações de *heap* acumuladas durante a execução. O valor reportado representa alocações cumulativas, não o pico de memória em uso simultâneo.

A reprodutibilidade dos *datasets* aleatórios é garantida por semente fixa no gerador pseudoaleatório (`rand.NewSource(42)`), assegurando que todos os experimentos recebam exatamente os mesmos dados de entrada em qualquer execução.

4.3 Design do Experimento

Cada combinação (algoritmo $\times$ tipo de *dataset* $\times$ tamanho de entrada) foi executada três vezes de forma independente. Os tipos de *dataset* e sua correspondência com os casos teóricos são apresentados na Tabela 2.

Table 2. Tipos de *dataset* e correspondência com casos teóricos.

Dataset	Descrição	Caso teórico
`ordenado`	$[1, 2, 3, \ldots, n]$	Melhor caso (Quick Sort)
`invertido`	$[n, n-1, \ldots, 1]$	Pior caso sem pivô mediana
`aleatório`	Permutação com seed fixa 42	Caso médio

Os tamanhos de entrada avaliados foram: 300.000, 450.000, 750.000, 1.000.000 e 2.500.000 elementos. Para cada grupo (algoritmo $\times$ *dataset*), foram calculadas as seguintes estatísticas a partir das três execuções: mínimo, máximo, média aritmética e desvio padrão.

4.4 Métricas Coletadas

As métricas registradas por execução foram: (i) duração em nanossegundos e milissegundos; (ii) número de comparações elemento-a-elemento; (iii) número de movimentações de elementos (trocas, deslocamentos ou distribuições para o Radix Sort); e (iv) alocações de *heap* em kilobytes. Os resultados foram exportados para arquivos CSV e processados com a biblioteca *pandas* em Python para geração de gráficos (*matplotlib*/*seaborn*) e tabelas estatísticas.

5 Resultados

5.1 Operações Elementares

A Tabela 3 apresenta o número de comparações, movimentações de elementos e alocações de memória para $n = 750.000$ elementos. Os valores são determinísticos e idênticos nas três execuções de um mesmo grupo. Para o Radix Sort, a coluna "Movimentações" contabiliza distribuições de elementos entre baldes por passagem ($k \times n$), não trocas entre posições do vetor — distinção fundamental que reflete a natureza não-comparativa do algoritmo.

5.2 Tempo de Execução

A Tabela 4 apresenta o tempo médio de execução em milissegundos para todos os cinco tamanhos de entrada avaliados. Os valores correspondem à média das três execuções

Table 3. Operações elementares para $n = 750.000$ elementos.

Algoritmo	Dataset	Comparações	Movimentações	Mem. (KB)
Quick Sort	ordenado	15.451.423	0	0
Quick Sort	invertido	15.451.424	375.000	0
Quick Sort	aleatório	20.874.735	3.396.991	0
Merge Sort	ordenado	7.180.608	14.701.424	118.350
Merge Sort	invertido	7.520.816	14.701.424	118.345
Merge Sort	aleatório	13.704.702	14.701.424	118.345
Heap Sort	ordenado	27.735.697	14.563.298	0
Heap Sort	invertido	26.357.656	13.416.716	0
Heap Sort	aleatório	27.009.663	13.993.716	0
Radix Sort	ordenado	0	4.500.000[a]	73.501
Radix Sort	invertido	0	4.500.000[a]	73.501
Radix Sort	aleatório	0	5.250.000[a]	79.365

[a] Radix Sort LSD base 10: ordenado/invertido = 6 dígitos $\times$ 750.000; aleatório = 7 dígitos $\times$ 750.000.

independentes.

As Figuras 1, 2 e 3 apresentam os tempos médios de execução em escala logarítmica (base 10) para entradas de 300.000, 750.000 e 2.500.000 elementos, respectivamente. A escala logarítmica é necessária para a visualização simultânea de algoritmos cujos tempos diferem em até uma ordem de grandeza para o mesmo tamanho de entrada.

6 Análise e Discussão

6.1 Confirmação Empírica das Complexidades Teóricas

Os resultados obtidos confirmam as predições teóricas de complexidade para os três algoritmos comparativos. Para o Merge Sort com entrada aleatória e $n = 750.000$, o número de comparações observado foi 13.704.702, valor próximo a $n \log_2 n \approx 14.640.000$ — razão de $0{,}94$, consistente com o comportamento esperado de $\Theta(n \log n)$ [5].

O Heap Sort apresentou contagens de comparações praticamente constantes entre os três tipos de *dataset*: 27.735.697 (ordenado), 26.357.656 (invertido) e 27.009.663 (aleatório) para $n = 750.000$. Esse comportamento decorre da invariante do algoritmo: as operações de *heapify* são determinadas estruturalmente pelo tamanho do *heap*, independentemente da distribuição inicial dos elementos. O valor observado de aproximadamente $1{,}84 \cdot n \log_2 n$ reflete o custo de duas comparações por nível na restauração do *heap*.

Para o Quick Sort com seleção de pivô pela mediana de três, as comparações em entrada ordenada foram 15.451.423 ($\approx 1{,}05 \cdot n \log_2 n$), evidenciando que a estratégia de

Table 4. Tempo médio de execução (ms) por algoritmo, *dataset* e tamanho de entrada.

Algoritmo	Dataset	300k	450k	750k	1.000k	2.500k
Quick Sort	ordenado	4,407	7,543	11,620	15,799	42,589
Quick Sort	invertido	4,945	7,573	11,162	16,401	42,730
Quick Sort	aleatório	21,459	32,467	56,313	75,809	200,900
Merge Sort	ordenado	17,584	25,181	42,335	55,819	157,627
Merge Sort	invertido	15,798	23,732	38,298	52,705	145,406
Merge Sort	aleatório	33,376	50,503	83,298	112,393	297,669
Heap Sort	ordenado	27,942	28,583	44,543	61,510	164,793
Heap Sort	invertido	20,198	31,801	64,463	70,042	167,258
Heap Sort	aleatório	33,150	50,025	92,327	126,829	430,186
Radix Sort	ordenado	12,434	16,402	27,519	43,138	104,497
Radix Sort	invertido	11,612	17,082	27,197	40,540	99,828
Radix Sort	aleatório	14,403	19,450	31,063	47,279	111,373

pivô garante comportamento $O(n \log n)$ mesmo no tipo de entrada que levaria o Quick Sort ingênuo ao pior caso.

O Radix Sort realizou zero comparações em todos os cenários, confirmando empiricamente sua natureza não-comparativa.

6.2 Quick Sort: Inversão de Desempenho por Seleção de Pivô

Um dos achados mais relevantes deste estudo é o comportamento contra-intuitivo do Quick Sort em relação à distribuição de entrada. Para $n = 750.000$, o tempo médio em entrada *ordenada* foi de apenas 11,620 ms, enquanto o tempo médio em entrada *aleatória* foi de 56,313 ms — uma diferença de fator $4{,}85\times$, com a entrada ordenada sendo significativamente *mais rápida*.

Esse comportamento é explicado pela interação entre a estratégia de mediana de três e a estrutura do vetor de entrada. Em um vetor completamente ordenado, o elemento mediano entre o primeiro, o central e o último é sempre exatamente a mediana do subvetor, produzindo partições perfeitamente balanceadas em cada nível de recursão. O resultado é o menor número possível de comparações por nível e ausência de trocas (0 trocas para entrada ordenada versus 3.396.991 para aleatória). Em entrada aleatória, a mediana de três é uma boa aproximação, mas não garante particionamento perfeito, resultando em maior variabilidade e número total de operações.

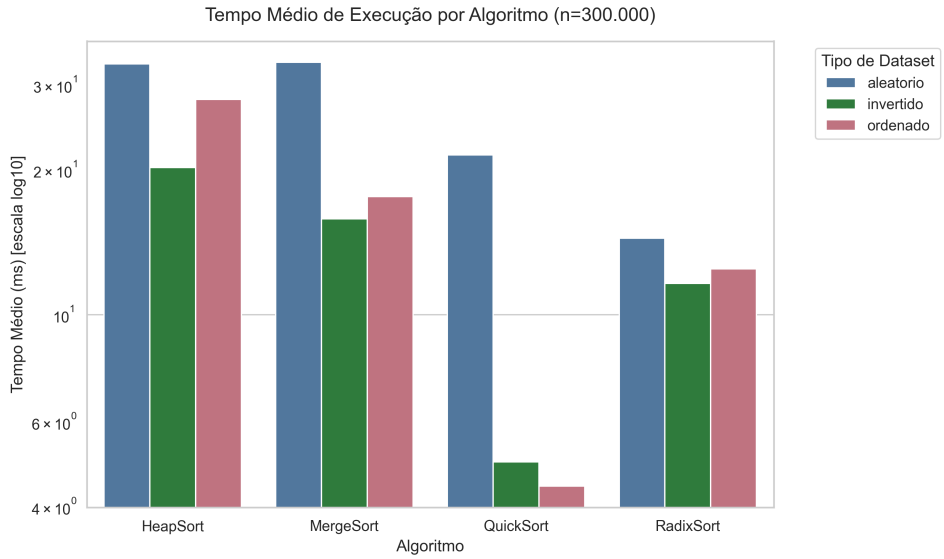

Figure 1. Tempo médio de execução para $n = 300.000$ elementos (escala $\log_{10}$).

O mesmo padrão é observado em entrada *invertida*: 11,162 ms para $n = 750.000$, também substancialmente mais rápido que o caso aleatório. Para $n = 2.500.000$, a diferença persiste: 42,589 ms (ordenado) e 42,730 ms (invertido) contra 200,900 ms (aleatório) — fator $4{,}71\times$. Esses resultados demonstram que, com seleção adequada de pivô, o Quick Sort atinge seu melhor desempenho prático justamente nos tipos de entrada que representam o pior caso da variante ingênua.

6.3 Radix Sort: Ruptura com o Limite Inferior Comparativo

O Radix Sort foi o algoritmo mais rápido em todos os tamanhos e tipos de *dataset* avaliados. Para $n = 750.000$ em entrada aleatória, o tempo médio foi de 31,063 ms, contra 56,313 ms do Quick Sort (fator $1{,}81\times$). Para $n = 2.500.000$, o Radix Sort executou em 111,373 ms contra 200,900 ms do Quick Sort (fator $1{,}80\times$), indicando que a vantagem relativa é estável com o crescimento de n.

A origem dessa vantagem é estrutural: o Radix Sort não está sujeito ao limite inferior $\Omega(n \log n)$ imposto à ordenação por comparação [2]. Com $k = 6$ passagens para entradas ordenadas e invertidas ($n \leq 10^6$, seis dígitos decimais) e $k = 7$ passagens para entradas aleatórias (valores aleatórios atingindo sete dígitos), a complexidade efetiva é $O(7n)$, linear no tamanho da entrada.

Uma consequência notável é a insensibilidade à distribuição de entrada: os tempos para ordenado (27,519 ms), invertido (27,197 ms) e aleatório (31,063 ms) diferem apenas em razão do número de dígitos do valor máximo (seis versus sete passagens). Em contraste, os algoritmos comparativos apresentam variações de até $4{,}85\times$ entre os tipos de *dataset*.

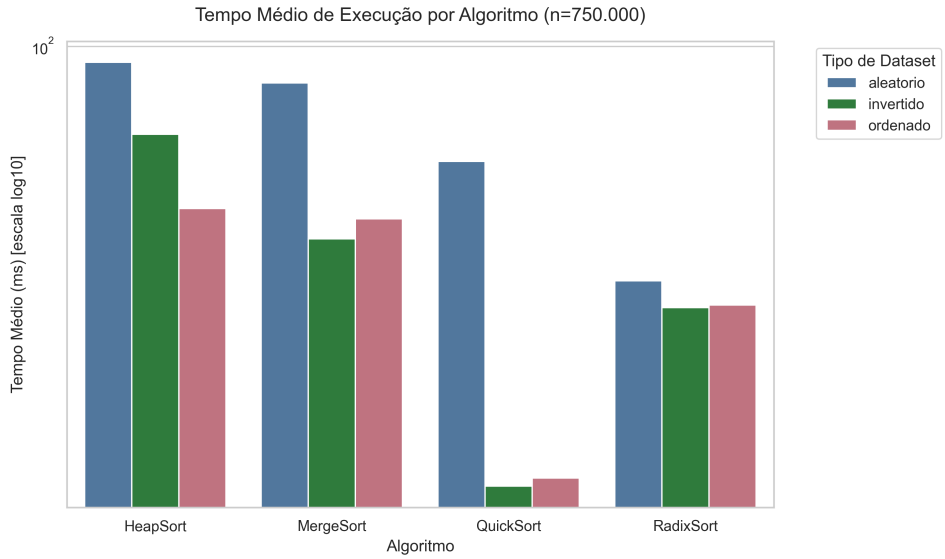

Figure 2. Tempo médio de execução para $n = 750.000$ elementos (escala $\log_{10}$).

O custo do Radix Sort é a memória auxiliar: 79.365 KB ($\approx$ 77 MB) para $n = 750.000$ em entrada aleatória, e 256.493 KB ($\approx$ 250 MB) para $n = 2.500.000$. Esse custo é significativo em sistemas embarcados ou ambientes com memória restrita, tornando o algoritmo inadequado em tais contextos mesmo sendo mais rápido.

6.4 Heap Sort: Custo Oculto da Desordem de Memória

O Heap Sort compartilha a classe de complexidade $O(n \log n)$ com Quick Sort e Merge Sort, porém apresentou consistentemente os maiores tempos de execução entre os algoritmos comparativos. Para $n = 750.000$ em entrada aleatória, o Heap Sort consumiu 92,327 ms, contra 83,298 ms do Merge Sort e 56,313 ms do Quick Sort — fatores $1{,}11\times$ e $1{,}64\times$, respectivamente. Para $n = 2.500.000$, o gap se amplia: 430,186 ms (Heap Sort) contra 200,900 ms (Quick Sort), fator $2{,}14\times$.

Essa degradação relativa está associada ao padrão de acesso à memória inerente à estrutura do *heap* binário. Cada operação de *sift-down* percorre um caminho da raiz para uma folha, acessando posições i, $2i + 1$ e $2i + 2$ do vetor — um padrão de saltos que não favorece a hierarquia de *cache* dos processadores modernos [1].

A alta variabilidade observada em entrada invertida — desvio padrão de 17,320 ms para uma média de 64,463 ms (27%) em $n = 750.000$ — é consistente com padrões irregulares de *cache miss* induzidos pela dinâmica específica do *heapify* sobre esse tipo de entrada. O Heap Sort é, em contrapartida, o único algoritmo avaliado que combina complexidade $O(n \log n)$ garantida no pior caso com complexidade de espaço $O(1)$ — propriedade relevante em contextos onde memória é escassa e garantias de pior caso são essenciais.

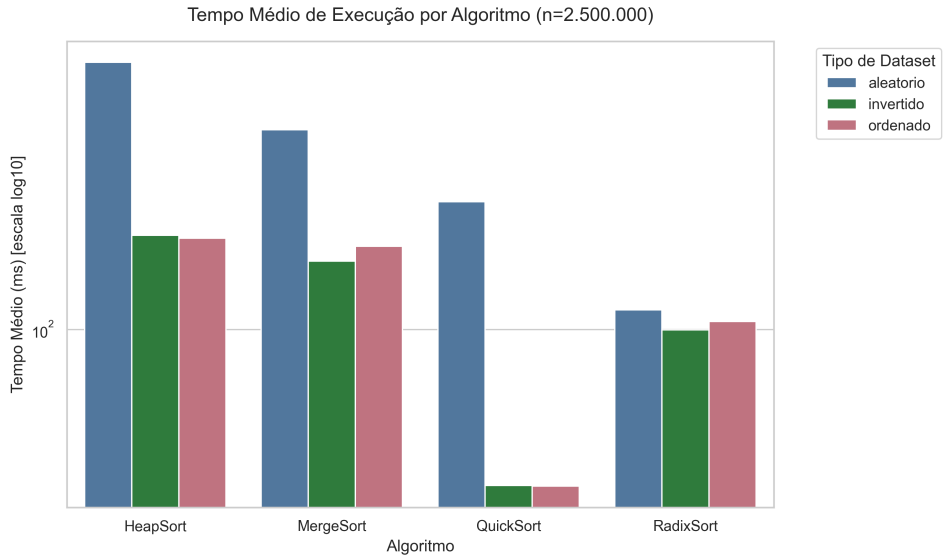

Figure 3. Tempo médio de execução para $n = 2.500.000$ elementos (escala $\log_{10}$).

6.5 Merge Sort: Estabilidade Temporal e Custo de Memória

O Merge Sort apresentou a menor variabilidade temporal entre todos os algoritmos avaliados. Para $n = 750.000$ em entrada aleatória, o desvio padrão foi de apenas 0,281 ms ($0{,}34\%$ da média de 83,298 ms), evidenciando comportamento altamente previsível — característica valiosa em sistemas de tempo real.

No entanto, a previsibilidade vem acompanhada de um custo de memória significativo. As alocações cumulativas de *heap* atingiram 118.350 KB ($\approx$ 116 MB) para $n = 750.000$ e 433.784 KB ($\approx$ 424 MB) para $n = 2.500.000$ — valores que podem ser proibitivos em sistemas embarcados. O padrão de alocações cumulativas, em lugar do pico de uso simultâneo, amplifica os valores reportados: cada operação de fusão aloca e libera um vetor auxiliar de tamanho proporcional ao subvetor processado, resultando em alocações totais de $O(n \log n)$.

O Merge Sort também apresentou o tempo médio mais baixo em entradas *invertidas* entre os algoritmos $O(n \log n)$ comparativos: 38,298 ms em $n = 750.000$, inferior ao resultado para entrada *ordenada* (42,335 ms). Essa inversão é atribuível à dinâmica das operações de fusão: para entradas invertidas, os subvetores de cada nível de recursão são sequências decrescentes perfeitamente regulares, permitindo fusões com padrão de acesso favorável ao *prefetch*.

6.6 Estabilidade das Medições

O desvio padrão das medições oferece uma estimativa da variabilidade introduzida pelo ambiente de execução. O algoritmo com maior instabilidade absoluta foi o Heap Sort em

$n = 2.500.000$ com entrada aleatória (desvio padrão de 20,968 ms, ou 4,9% da média de 430,186 ms). O Merge Sort, em contrapartida, atingiu desvio padrão de apenas 0,140 ms para $n = 1.000.000$ em entrada aleatória (0,12% da média).

O Quick Sort apresentou desvio padrão inferior a 1% em todos os cenários de entrada ordenada e invertida, e valores entre 0,4% e 1,6% para aleatória, confirmando comportamento estável com a estratégia de mediana de três. O Radix Sort manteve desvio padrão abaixo de 3% em todos os cenários, consistente com seu comportamento determinístico por passagem.

7 Conclusão

Este trabalho apresentou uma avaliação empírica e reprodutível de quatro algoritmos de ordenação sobre entradas de até 2.500.000 elementos. Os experimentos confirmaram quantitativamente as predições da análise assintótica e revelaram distinções práticas relevantes que a notação $O(\cdot)$ não captura.

Os principais achados podem ser sintetizados em quatro constatações. Em primeiro lugar, o Radix Sort supera consistentemente todos os algoritmos comparativos por contornar o limite inferior $\Omega(n \log n)$ — com fator $1{,}8\times$ em relação ao Quick Sort para $n = 2.500.000$ aleatório — ao custo de memória auxiliar significativa (≈ 250 MB para 2.500.000 elementos). Em segundo lugar, a estratégia de pivô determina o comportamento prático do Quick Sort de forma mais significativa que o tipo de entrada: com mediana de três, entradas ordenadas e invertidas são processadas $4{,}7\times$ mais rápido que entradas aleatórias. Em terceiro lugar, algoritmos de mesma classe $O(n \log n)$ podem divergir em até $2{,}1\times$ (Heap Sort versus Quick Sort em $n = 2.500.000$), com o Heap Sort penalizado pela baixa localidade de *cache* apesar de ser o único algoritmo in-place com pior caso garantido. Em quarto lugar, o Merge Sort entrega a maior estabilidade temporal (desvio padrão de 0,14 ms em $n = 1.000.000$), mas impõe o maior custo de memória cumulativa entre os algoritmos avaliados.

Este trabalho apresenta algumas limitações. Os experimentos foram conduzidos em um único hardware com arquitetura x86-64; comportamentos de *cache* e *pipeline* podem diferir em processadores ARM ou em ambientes com hierarquias de memória distintas. O Radix Sort foi avaliado apenas para inteiros positivos de até sete dígitos decimais; sua extensão a inteiros negativos ou a chaves de comprimento variável requereria adaptação da implementação. As alocações de memória reportadas são cumulativas, não representando o pico de uso simultâneo — métrica complementar relevante para análise de pressão de memória.

Como trabalhos futuros, pretende-se avaliar algoritmos híbridos de produção como Introsort e Timsort, que combinam Quick Sort, Heap Sort e Insertion Sort para garantir $O(n \log n)$ no pior caso com constante prática mínima. Adicionalmente, planeja-se estender a análise ao Counting Sort e ao Bucket Sort para dados de distribuição uniforme, e conduzir testes estatísticos formais de comparação de médias (Wilcoxon signed-rank) para validar a significância estatística das diferenças observadas.

References

[1] Paul Biggar, Nicholas Nash, Kevin Williams, and David Gregg. An experimental study of sorting and branch prediction. *ACM Journal of Experimental Algorithmics*, 12:1.8:1–1.8:39, 2008.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, 3 edition, 2009.

[3] Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. *Data Structures and Algorithms in Java*. Wiley, Hoboken, 6 edition, 2014.

[4] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.

[5] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, 2 edition, 1998.

[6] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.

[7] Galo Octavio Pizarro-Vasquez, Franklin Mejia Morales, Pietro Galvez Minervini, and Miguel Botto-Tobar. Sorting algorithms and their execution times: An empirical evaluation. In *Technology Trends — ICAETT 2020*, volume 1302 of *Advances in Intelligent Systems and Computing*, Cham, 2020. Springer.

[8] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley Professional, Upper Saddle River, 4 edition, 2011.

[9] J. W. J. Williams. Algorithm 232 (HEAPSORT). *Communications of the ACM*, 7(6):347–348, 1964.